Bài 2.14 – DBSCAN: phân cụm theo mật độ

> **Giai đoạn 2 – Machine Learning cổ điển** · Thời lượng: 1 buổi (2–3 giờ) · Hình thức mở đầu: *Tìm lỗi*

🎯 Sau bài này bạn sẽ

- Hiểu DBSCAN: cụm là **vùng dày đặc**, ngăn cách bởi **vùng thưa**.
- Phân biệt 3 loại điểm: **lõi**, **biên**, **nhiễu**, và tự chạy tay trên ví dụ nhỏ.
- Tìm được cụm hình dạng bất kỳ, **không cần chọn K**, và tự động **phát hiện điểm bất thường**.
- Chọn tham số `eps` bằng biểu đồ k-khoảng cách.
- Tìm các **vùng đô thị** của California chỉ từ tọa độ.

---

1. Tìm lỗi: K-Means thất bại 🐞

Một bạn dùng K-Means (bài 2.13) để phân cụm dữ liệu hai mặt trăng lồng nhau (bài 2.12):

```
import numpy as np
import pandas as pd
from sklearn.datasets import make_moons
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score

X, y = make_moons(n_samples=300, noise=0.07, random_state=42)   # y: nhãn thật, chỉ dùng để chấm điểm
km = KMeans(n_clusters=2, n_init=10, random_state=42).fit(X)
print("K-Means ARI:", round(adjusted_rand_score(y, km.labels_), 3))
```

Kết quả mẫu:

```
K-Means ARI: 0.254
```

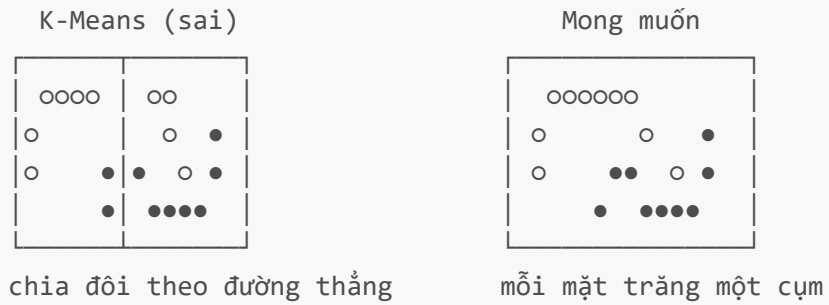
(**ARI – Adjusted Rand Index:** đo mức khớp giữa cụm tìm được và nhóm thật. 1 = khớp hoàn hảo, 0 ≈ chia ngẫu nhiên. Không quan tâm số hiệu cụm.)

Code không có lỗi cú pháp, K = 2 cũng đúng. **Lỗi nằm ở đâu?**

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

Lỗi nằm ở **lựa chọn thuật toán**. K-Means gán mỗi điểm về **tâm gần nhất**, nên ranh giới giữa hai cụm luôn là một **đường thẳng** ở giữa hai tâm. Nó cắt đôi mỗi mặt trăng theo chiều dọc thay vì tách hai mặt trăng ra:



Cần một thuật toán hiểu cụm là **"vùng điểm nối liền nhau"**, không phải "điểm quanh một tâm".

---

2. Ý tưởng DBSCAN

2.1. Khái niệm

- **DBSCAN – Density-Based Spatial Clustering of Applications with Noise (Phân cụm theo mật độ, có xử lý nhiễu):** Cụm là những vùng mà các điểm **đứng sát nhau**. Điểm đứng lẻ loi ở vùng thưa là **nhiễu**.
- **Ví dụ đời thực:** Nhìn ảnh vệ tinh ban đêm. Các vùng sáng dày đặc là **thành phố** (cụm), dù hình dạng méo mó thế nào. Những đốm sáng lẻ loi giữa vùng tối là **nhà đơn lẻ** (nhiễu).

2.2. Hai tham số

Tham số	Ý nghĩa
`eps` (ε)	Bán kính "hàng xóm": hai điểm cách nhau ≤ eps là hàng xóm
`min_samples`	Số hàng xóm tối thiểu (**tính cả chính nó**) để một điểm được coi là nằm trong vùng dày đặc

2.3. Ba loại điểm



Thuật toán:

1. Tìm mọi **điểm lõi**.
2. Các điểm lõi là hàng xóm của nhau được **nối** vào cùng một cụm (lan dần như vết dầu loang).

3. Điểm biên được gán vào cụm của điểm lõi gần nó.
4. Điểm còn lại là **nhiễu**, nhãn **−1**.

2.4. Chạy tay trên trục số

8 điểm: **1, 2, 3, 4, 10, 11, 12, 25**. eps = 1,5; min_samples = 3.

Điểm	Hàng xóm trong khoảng ±1,5 (tính cả chính nó)	Số lượng	Loại
1	1, 2	2	Không phải lõi; nhưng gần lõi 2 → **biên**
2	1, 2, 3	3	**Lõi**
3	2, 3, 4	3	**Lõi**
4	3, 4	2	Gần lõi 3 → **biên**
10	10, 11	2	Gần lõi 11 → **biên**
11	10, 11, 12	3	**Lõi**
12	11, 12	2	Gần lõi 11 → **biên**
25	25	1	Không gần lõi nào → **nhiễu**

- Lõi 2 và 3 là hàng xóm → cùng cụm 0, kéo theo biên 1 và 4: cụm 0 = {1, 2, 3, 4}.
- Lõi 11 một mình, kéo theo 10 và 12: cụm 1 = {10, 11, 12}.
- 25 là nhiễu.

```
from sklearn.cluster import DBSCAN

x1 = np.array([1, 2, 3, 4, 10, 11, 12, 25]).reshape(-1, 1)
d1 = DBSCAN(eps=1.5, min_samples=3).fit(x1)
print(d1.labels_)                  # nhãn cụm; -1 = nhiễu
print(d1.core_sample_indices_)     # vị trí các điểm lõi
```

Kết quả mẫu:

```
[ 0  0  0  0  1  1  1 -1]
[1 2 5]
```

Khớp hoàn toàn: điểm lõi ở vị trí 1, 2, 5 (giá trị 2, 3, 11). **Không ai bảo DBSCAN có 2 cụm**: nó tự tìm ra.

---

3. Giải bài hai mặt trăng

```
db = DBSCAN(eps=0.2, min_samples=5).fit(X)
print("Các nhãn:", sorted(set(db.labels_)),
      "| ARI:", round(adjusted_rand_score(y, db.labels_), 3),
      "| số điểm nhiễu:", (db.labels_ == -1).sum())
```

Kết quả mẫu:

```
Các nhãn: [0, 1] | ARI: 1.0 | số điểm nhiễu: 0
```

ARI = **1,0**: tách hai mặt trăng **hoàn hảo** (so với 0,254 của K-Means). DBSCAN "đi dọc" theo từng mặt trăng vì các điểm trên đó nối liền nhau.

Ảnh hưởng của eps

```
for eps in [0.05, 0.1, 0.2, 0.3, 0.5]:
    d = DBSCAN(eps=eps, min_samples=5).fit(X)
    so_cum = len(set(d.labels_)) - (1 if -1 in d.labels_ else 0)   # không tính
nhãn -1
    print(f"eps = {eps}: số cụm = {so_cum} | số điểm nhiễu = {(d.labels_ ==
-1).sum()}")
```

Kết quả mẫu:

```
eps = 0.05: số cụm = 3 | số điểm nhiễu = 282
eps = 0.1: số cụm = 11 | số điểm nhiễu = 47
eps = 0.2: số cụm = 2 | số điểm nhiễu = 0
eps = 0.3: số cụm = 2 | số điểm nhiễu = 0
eps = 0.5: số cụm = 1 | số điểm nhiễu = 0
```

eps	Hiện tượng
Quá nhỏ (0,05)	Gần như mọi điểm đều "cô đơn" → 282/300 điểm là nhiễu
Hơi nhỏ (0,1)	Mặt trăng bị vỡ thành 11 mảnh
Vừa (0,2–0,3)	Đúng 2 cụm
Quá lớn (0,5)	Hai mặt trăng bị nối vào nhau thành 1 cụm

DBSCAN **rất nhạy** với eps. Cần một cách chọn có cơ sở.

---

4. Chọn eps bằng biểu đồ k-khoảng cách

Ý tưởng: Với mỗi điểm, đo khoảng cách tới hàng xóm thứ k (k = min_samples). Điểm trong vùng dày có khoảng cách nhỏ, điểm nhiễu có khoảng cách lớn. Sắp xếp tăng dần và vẽ: chỗ đường cong **bắt đầu dựng đứng** là giá trị eps hợp lý.

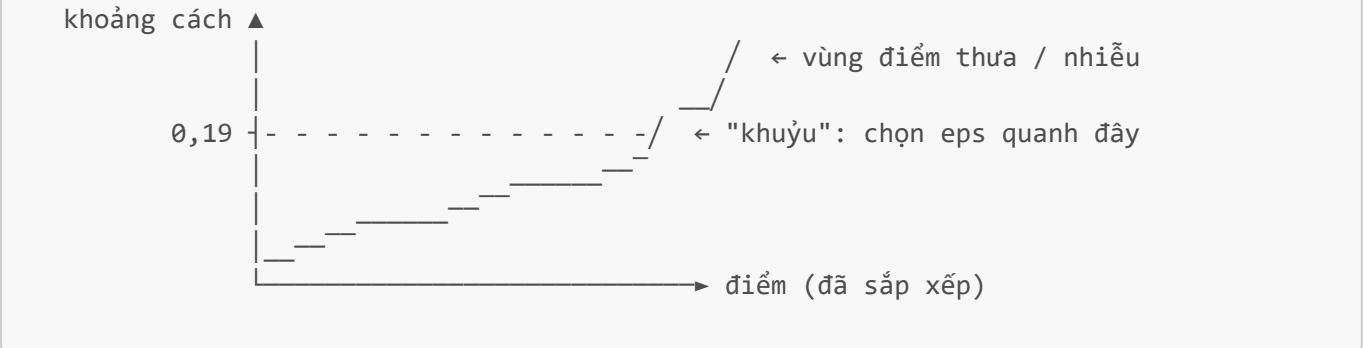
```
import matplotlib.pyplot as plt
from sklearn.neighbors import NearestNeighbors

nn = NearestNeighbors(n_neighbors=5).fit(X)        # 5 = min_samples (tính cả chính nó)
kc, _ = nn.kneighbors(X)                           # khoảng cách tới 5 hàng xóm gần nhất
k_dist = np.sort(kc[:, -1])                        # lấy hàng xóm thứ 5, sắp tăng dần
print(np.percentile(k_dist, [50, 90, 95, 99]).round(3))
plt.plot(k_dist); plt.ylabel("Khoảng cách tới hàng xóm thứ 5"); plt.xlabel("Điểm (đã sắp xếp)")
plt.show()
```

Kết quả mẫu:

```
[0.089 0.139 0.159 0.186]
```

Bạn sẽ thấy: đường cong đi lên chậm rồi **cong vút lên** ở cuối. 99% số điểm có hàng xóm thứ 5 trong vòng 0,186 → chọn eps khoảng **0,15–0,2**, khớp với kết quả thử ở mục 3.



Chọn min_samples: quy tắc kinh nghiệm: ít nhất bằng (số chiều + 1); với dữ liệu nhiều nhiễu thì tăng lên (thường 5–20; dữ liệu lớn có thể hàng chục).

---

5. Ứng dụng: tìm vùng đô thị California

Chỉ dùng **tọa độ** các khu dân cư, DBSCAN có tìm ra các thành phố không?

Lưu ý kỹ thuật: khoảng cách trên Trái Đất không phải đường thẳng trên mặt phẳng. Dùng `metric="haversine"` (khoảng cách trên mặt cầu), tọa độ đổi sang **radian**, và eps tính bằng km chia cho bán kính Trái Đất (6.371 km).

```
df = nap_du_lieu_nha()                                  # ô chuẩn bị dữ liệu ở
bài 2.1
toa_do = np.radians(df[["vi_do", "kinh_do"]].values)     # haversine yêu cầu [vĩ
độ, kinh độ] theo radian

for eps_km, ms in [(5, 30), (10, 50)]:
    db = DBSCAN(eps=eps_km / 6371, min_samples=ms, metric="haversine",
algorithm="ball_tree").fit(toa_do)
    lab = db.labels_
    so_cum = len(set(lab)) - (1 if -1 in lab else 0)
    print(f"eps = {eps_km} km, min_samples = {ms}: số cụm = {so_cum} | nhiễu =
{(lab == -1).sum()} ({(lab == -1).mean():.1%})")
```

Kết quả mẫu:

```
eps = 5 km, min_samples = 30: số cụm = 42 | nhiễu = 3442 (21.3%)
eps = 10 km, min_samples = 50: số cụm = 21 | nhiễu = 2796 (17.3%)
```

Xem các cụm lớn nhất (eps = 5 km):

```
db = DBSCAN(eps=5 / 6371, min_samples=30, metric="haversine",
algorithm="ball_tree").fit(toa_do)
df["cum"] = db.labels_
thong_ke = df.groupby("cum").agg(so_khu=("gia_nha", "size"), vi_do=("vi_do",
"mean"),
                                 kinh_do=("kinh_do", "mean"), gia_tb=("gia_nha",
"mean")).round(2)
print(thong_ke.sort_values("so_khu", ascending=False).head(8))
```

Kết quả mẫu:

```
     so_khu  vi_do  kinh_do  gia_tb
cum
 7     5531  33.98  -118.15  221.54
-1     3442  36.87  -120.07  145.80
 27    1351  37.50  -122.16  294.38
 31     927  37.78  -122.19  200.00
 0      874  32.74  -117.09  176.56
 23     566  38.61  -121.39  129.19
 3      450  34.05  -117.34  116.75
 15     284  36.78  -119.78   87.30
```

Đối chiếu tọa độ với bản đồ, các cụm chính là:

Cụm	Vị trí (vĩ độ, kinh độ)	Vùng đô thị	Giá TB (nghìn USD)
7	33,98; −118,15	**Vùng Los Angeles** (siêu đô thị nối liền)	221,5
27	37,50; −122,16	**Bán đảo San Francisco – Thung lũng Silicon**	294,4
31	37,78; −122,19	**Oakland – bờ đông vịnh SF**	200,0
0	32,74; −117,09	**San Diego**	176,6
23	38,61; −121,39	**Sacramento** (thủ phủ bang)	129,2
3	34,05; −117,34	**Riverside – San Bernardino**	116,8
15	36,78; −119,78	**Fresno** (vùng nông nghiệp trung tâm)	87,3
−1	(rải rác)	**Nông thôn, thị trấn nhỏ**	145,8

DBSCAN **tự** tìm ra các vùng đô thị lớn của California, kể cả tách được bán đảo SF và Oakland ở hai bờ vịnh (vì nước biển ở giữa là "vùng thưa"). K-Means không làm được việc này: nó sẽ cố chia đều thành các "vùng tròn" và gán cả những khu nông thôn lẻ loi vào một thành phố nào đó.

Ứng dụng tiếp theo: cột `cum` (hoặc "có thuộc đô thị lớn không") có thể làm **đặc trưng mới** cho mô hình giá nhà (bài 2.16).

---

6. DBSCAN để phát hiện bất thường

Điểm nhiễu (−1) chính là những điểm **không thuộc vùng đông đúc nào**: ứng viên cho **bất thường (anomaly)**.

- Giao dịch thẻ ở vị trí lạ, số tiền lạ so với thói quen.
- Cảm biến máy móc báo trị số khác hẳn bình thường.
- Đơn hàng có tổ hợp sản phẩm, giờ đặt, địa chỉ bất thường.

> 💡 Phiên bản mới **HDBSCAN** (`sklearn.cluster.HDBSCAN`, có từ Scikit-learn 1.3) tự xử lý được cụm có **mật độ khác nhau** và ít nhạy với eps hơn. Đáng thử khi DBSCAN khó chỉnh.

---

7. So sánh K-Means và DBSCAN

	K-Means	DBSCAN
Cần chọn số cụm?	**Có** (K)	Không
Hình dạng cụm	Tròn, cùng cỡ	**Bất kỳ**
Xử lý nhiễu / outlier	Mọi điểm đều bị gán; outlier kéo lệch tâm	**Tự đánh dấu nhiễu (−1)**
Tham số khó chọn	K	eps (rất nhạy), min_samples
Cụm mật độ khác nhau	Được	**Kém** (một eps không hợp cho mọi mật độ)

	K-Means	DBSCAN
Gán điểm mới (`predict`)	Có	**Không có** sẵn
Tốc độ với dữ liệu lớn	Nhanh	Chậm hơn
Cần chuẩn hóa	Có	Có (dựa trên khoảng cách)

8. Bài tập

1. **Chạy tay** DBSCAN trên các điểm **1, 2, 3, 4, 10, 11, 12, 25** với eps = 1,5 nhưng **min_samples = 2**. Kết quả khác gì so với mục 2.4?
2. Với dữ liệu hai mặt trăng, đặt eps = 0,2 và thử min_samples = 3, 10, 30. Chuyện gì xảy ra?
3. Ở mục 5, thử eps = 2 km, min_samples = 30. Số cụm và tỉ lệ nhiễu thay đổi thế nào? Vùng LA có còn là một cụm không?
4. Tạo đặc trưng `o_do_thi = (df["cum"] != -1).astype(int)` từ mục 5. Giá trung bình của khu đô thị và khu nhiễu chênh nhau bao nhiêu?

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. Với min_samples = 2, mọi điểm có ít nhất 1 hàng xóm khác đều là lõi: 1, 2, 3, 4, 10, 11, 12. Cụm 0 = {1, 2, 3, 4}, cụm 1 = {10, 11, 12}, 25 vẫn là nhiễu (không có hàng xóm). Kết quả cụm giống hệt, nhưng **nhiều điểm lõi hơn** (không còn điểm biên).

```
print(DBSCAN(eps=1.5, min_samples=2).fit(x1).labels_)   # [ 0  0  0  0  1  1  1
-1]
```

2. min_samples nhỏ → dễ thành lõi, ít nhiễu. min_samples lớn (30) với eps = 0,2 → khó đủ hàng xóm → **nhiều điểm thành nhiễu**, mặt trăng có thể bị vỡ. eps và min_samples phải chọn cùng nhau: tăng min_samples thì thường phải tăng eps.
3. Kết quả thật với eps = 2 km: **20 cụm**, nhưng **78,5%** số khu bị coi là nhiễu! Cụm lớn nhất chỉ còn 1.195 khu: siêu đô thị LA (5.531 khu ở eps = 5 km) bị vỡ vụn, chỉ còn những lõi dày đặc nhất được giữ lại. eps quá nhỏ so với mật độ dữ liệu → hầu hết điểm không đủ hàng xóm.
4. Code:

```
df["o_do_thi"] = (df["cum"] != -1).astype(int)
print(df.groupby("o_do_thi")["gia_nha"].mean().round(1))
```

Kết quả: khu nhiễu (0) có giá TB **145,8**, khu đô thị (1) **205,2** nghìn USD, chênh khoảng 60.000 USD. Đặc trưng này tóm tắt "có nằm trong vùng đô thị dày đặc không", và có thể giúp mô hình ở Project 1.

9. Flashcard

Mặt trước	Mặt sau
DBSCAN định nghĩa cụm là gì?	Vùng điểm dày đặc nối liền nhau, ngăn cách bởi vùng thưa.
Hai tham số của DBSCAN?	eps (bán kính hàng xóm), min_samples (số hàng xóm tối thiểu, tính cả chính nó).
Điểm lõi là gì?	Điểm có ≥ min_samples hàng xóm trong bán kính eps.
Điểm biên là gì?	Không đủ hàng xóm nhưng nằm trong vùng của một điểm lõi.
Nhãn −1 nghĩa là?	Nhiễu: không thuộc cụm nào.
DBSCAN có cần chọn số cụm?	Không.
Chọn eps thế nào?	Biểu đồ k-khoảng cách: chọn chỗ đường cong dựng đứng.
eps quá nhỏ / quá lớn?	Nhỏ: nhiều nhiễu, cụm vỡ vụn. Lớn: các cụm dính vào nhau.
Điểm yếu của DBSCAN?	Nhạy với eps; kém khi các cụm có mật độ khác nhau; không có `predict`.
Khoảng cách cho tọa độ GPS?	Haversine (tọa độ theo radian, eps = km / 6371).

Bài tiếp theo: 2.15 – PCA.